

概述

什么是软件架构？

- 程序或计算系统的软件架构是该系统的结构（一个或多个），这些结构由软件元素、这些元素的外部可见属性以及它们之间的关系组成。
- 系统的基础组织，体现在其组件、组件之间以及组件与环境的关系中，以及指导其设计和演化的原则。

Architecture vs. Design

1. 所有架构都是软件设计，但并非所有设计都是软件架构
2. “架构设计”是设计过程的一部分
3. 架构是更高层级的，更倾向于黑盒设计

Architecture vs. Structure

架构定义了：

1. 组件接口：组件能做什么？
2. 组件通信
3. 组件职责：覆盖功能需求和非功能需求

而结构没有

架构主要解决非功能需求，尤其是质量属性

架构规定了通信方式

通信包含

1. 数据传递机制：函数调用、远程方法调用、异步消息
2. 控制流：顺序、并发、同步

架构解决非功能需求（NFRs）（NFRs 又称架构需求）

NFRs 包括：

1. 技术约束
2. 业务约束
3. 质量属性

设计是一种抽象

架构提供了设计的高层抽象视图。

- 隐藏了设计的复杂性和实现细节
- 架构元素与软件元素之间可能存在直接映射，也可能不存在

黑盒设计与白盒设计

- 对系统结构和交互的非正式描述。
- 描绘架构中蕴含的设计理念

架构视图

如何进行设计开发？ how to develop a design?

1. 分解
2. 抽象
3. 分而治之
4. 生成并测试
5. 迭代：增量细化
6. 可复用元素

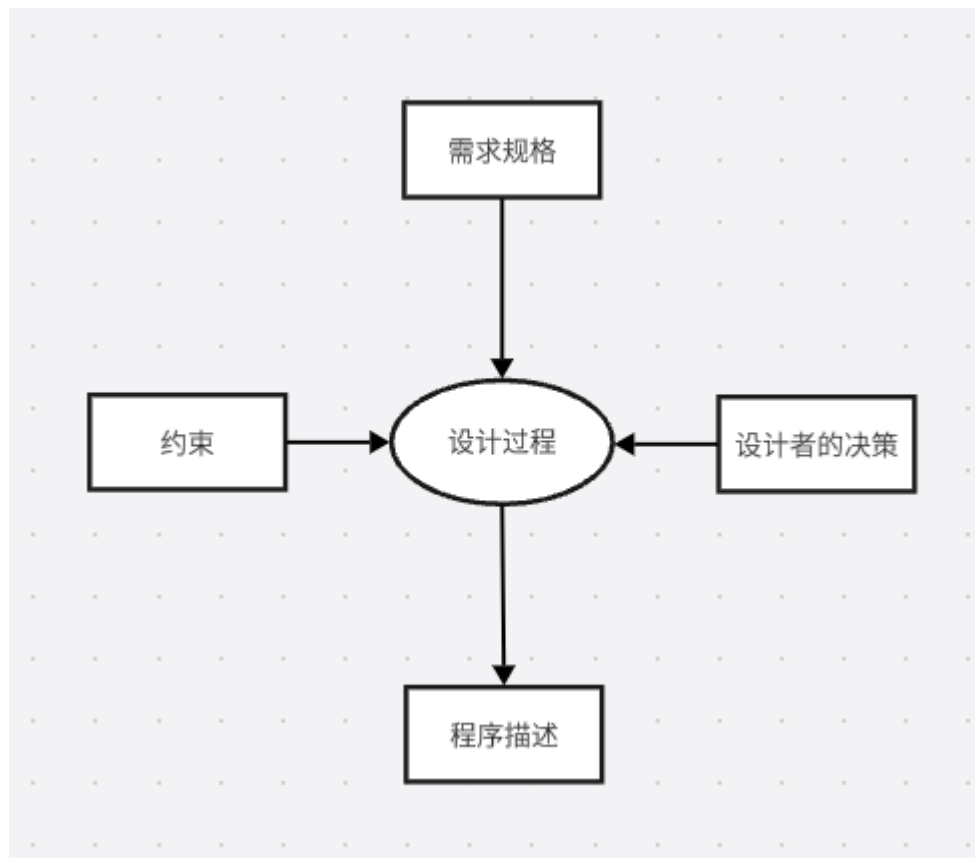
P. Kruchten 的4 + 1视图模型

1. 逻辑视图：描述架构中具有架构显著性的元素以及它们之间的关系
2. 进程视图：描述架构中的并发和通信
3. 物理视图：描绘主要进程和组件如何映射到应用程序的硬件上
4. 开发视图：捕获软件组件的内部组织，例如保存在配置管理工具中的组件
5. 架构用例：捕获架构的需求；与多个特定视图相关

架构师做什么？ What Does a Software Architect Do?

1. 联络沟通
2. 软件工程
3. 技术知识
4. 风险管理

通用设计模型 general design model



软件设计过程

需求规格说明 -> 架构设计决策 -> 逻辑设计细节 -> 详细设计决策 -> 物理设计细节

架构活动 Architecture Activities

1. 为系统创建业务案例
2. 理解需求
3. 创建并选择架构
4. 沟通架构
5. 分析或评估架构
6. 实现架构
7. 确保符合架构

架构生命周期 lifecycle

1. 架构构建
2. 架构分析
3. 架构评估
4. 架构实现
5. 架构维护

为什么软件架构很重要？

功能性需求

1. 功能需求说明了系统**必须做什么**，并阐述了系统如何为利益相关者提供价值。
2. 功能需求指的是系统的**行为**。
3. 功能性是系统执行其预期工作的能力，例如，允许学生在线注册。
4. 功能性可以通过任意数量的可能结构来实现。
5. 功能性在很大程度上独立于结构，因为它可以作为一个没有任何内部结构的单一单体系统存在。

质量需求

1. 质量需求是整体系统所期望的特性（即质量属性），系统应在功能需求之上提供这些特性。
2. 质量需求是对功能需求或整体产品的限定。
3. 如果质量属性很重要，那么软件架构会约束功能到各种结构上的分配（映射）。

约束

1. 约束是一个**自由度为零**的设计决策。
2. 约束是预先指定好的、已经做出的设计决策。
3. 通过接受该设计决策并将其与其他受影响的设计决策进行协调，即可满足约束。

质量需求

非功能需求

- 非功能需求或架构需求是质量属性的替代术语
- 不可能先确保功能正确，然后再试图适应非功能需求
- **两个类别**
 1. **执行时可观察的（外部）**：系统在多大程度上满足其行为需求？例如，性能、安全性、可用性、易用性等
 2. **执行时不可观察的（内部）**：系统被维护、集成或测试的容易程度如何？例如，可修改性、可移植性、可重用性、可测试性等

定义质量属性

- 质量属性场景被用来定义所需的质量属性
- 场景分成两个类别：
 - **通用场景**：是与系统无关的场景，用于指导质量属性需求的规范。
 - **具体场景**：是与特定系统相关的场景，用于指导特定系统的质量属性需求规范。它们是通用场景的实例。

通用场景建模

1. 刺激源
2. 刺激
3. 制品
4. 环境
5. 响应
6. 响应度量

战术

- 战术是一种设计决定
- 战术的集合称为架构策略

质量设计决定

- 架构是设计决定的集合

七类：

1. 责任分配
2. 协调模型
3. 数据模型
4. 资源管理
5. 架构元素间的映射
6. 绑定时间决策
7. 技术选择

质量属性与战术

可用性

...

ASR (architecturally significant requirements)

如何系统地识别将影响ASR和其他因素？

1. 从**需求文档**中收集ASR Gathering ASRs from requirements documents
2. 通过采访**涉众**来收集ASR Gathering ASRs by interviewing stakeholders
3. 通过了解**业务目标**来收集ASR Gathering ASRs by understanding the business goals
4. 在**项目树**中管理ASR Capturing ASRs in a utility tree